DIS · 만들어 사용하는 봇에서, 문제를 고쳐 가는 서비스로

Discord에서 음악을 듣고 주식 모의투자를 할 수 있는 Java 봇
윤영주 · 개발 및 운영 · 저장소 기록 2025.06~
프로젝트 GitHub

현재 지인 약 30명과 사용하고 있습니다.

1. 어떤 프로젝트인가요?

DIS는 Discord 채팅에서 음악을 검색하고 음성 채널로 들을 수 있는 봇입니다. 곡을 대기열에 넣거나 일시정지·건너뛰기 같은 조작을 할 수 있습니다. 이후 가상 자금으로 주식을 사고팔고, 계좌와 수익률 순위를 확인하는 모의투자 기능을 추가했습니다.

직접 만들게 된 계기는 2024년 Discord 봇들이 동작하지 않던 경험이었습니다. 사용하던 기능을 이용할 수 없게 되자, 필요한 음악 기능을 직접 만들어 사용해 보기로 했습니다. Java와 JDA, LavaPlayer로 기본 기능을 구현한 뒤 GCP에 올렸고, 현재는 지인 약 30명과 함께 사용하고 있습니다.

같은 해 공개 음악 라이브러리에서도 로그인 요구와 재생 실패가 보고됐습니다. 당시 상황을 확인한 관련 자료도 함께 정리했습니다. 2024년의 경험이 제작 계기가 되었고, 저장소에서 확인되는 첫 개발 기록은 2025년 6월입니다.

기능이 늘어나고 배포를 반복하면서, 코드를 실행하는 것과 사용자가 정상적으로 이용하는 것은 다르다는 점을 경험했습니다. 재생 안내는 나오는데 음악이 들리지 않거나, 기능 코드와 별개로 빌드가 막히는 상황이 있었습니다. 이 프로젝트에서는 그런 문제를 확인하고 고쳐 나가는 과정에 집중했습니다.

2. 사용자는 어떻게 이용하나요?

하고 싶은 일	사용 흐름
음악 듣기	음성 채널 입장 → `/play` 로 곡 검색 → 재생·대기열 확인
재생 조작	`/pause` , `/resume` , `/skip` , `/stop` 으로 제어
모의투자	시세 확인 → 가상 자금으로 매수·매도 → 계좌·거래 내역·순위 조회
사용법 확인	`/man` 으로 본인에게만 보이는 명령 안내 확인

음악 재생 안내나 계좌 조회처럼 개인에게 필요한 결과는 요청한 사람에게 보여 줍니다. 모의투자의 매매·순위처럼 함께 볼 성격의 결과는 채널에 공개합니다.

3. 기능을 늘리며 역할도 나눴습니다

현재는 명령을 받는 부분, 음악을 처리하는 부분, 주식 모의투자를 처리하는 부분을 나누어 실행합니다. 처음부터 이 구조였던 것은 아니며, 기능을 추가하고 응답 흐름을 정리하는 과정에서 바뀌었습니다.

01	02	03	04
명령 입력	**답변 준비**	**작업 처리**	**결과 안내**
Discord 사용자	돌려줄 위치를 먼저 기록	음악 또는 모의투자	원래 요청에 답변

작업을 주고받는 데 RabbitMQ를 사용하고, 응답을 기다리는 정보와 일부 상태는 Redis에, 모의투자 계좌와 거래 기록은 PostgreSQL에 저장합니다. 각 부분이 나누어져 있어도 **요청한 사람이 결과를 받을 때까지 하나의 흐름으로 이어져야 한다**는 점을 중요하게 생각했습니다.

핵심 챌린지 세 가지

1. “재생됐다고 나오는데 왜 소리가 안 날까?”

봇은 음성 채널에 들어오고 재생 안내도 보냈지만 실제로는 음악이 나오지 않았습니다. 연결 여부와 곡 검색 성공만 보지 않고, 음원 데이터를 가져오는 단계의 로그와 인증 상태를 확인했습니다. 외부 음원 요청 방식의 호환성을 보완해 재생을 복구했고, 요청을 받은 상태와 실제 재생 실패를 구분해 안내하도록 개선했습니다.

2. “작업이 끝났는데 답변할 사용자를 놓치면 안 된다.”

명령을 받는 부분과 음악을 처리하는 부분을 나누면서, 처리 결과를 원래 요청에 돌려주는 순서도 정리했습니다. 답변할 위치를 먼저 저장하고 작업을 보내도록 바꿨으며, 작업이 빨리 끝나도 이 순서가 지켜지는지 테스트했습니다.

3. “기능 코드를 바꾸지 않았는데 빌드가 막혔다.”

외부 라이브러리 저장소가 필요한 설명 파일 대신 웹페이지를 반환해 빌드가 실패한 문제에 대응했습니다. 원인을 확인한 뒤 같은 버전의 공식 배포 파일을 받도록 바꾸고 검증값을 확인했습니다. 변경 이후 자동 빌드와 서버 배포 성공을 확인하며, 서비스 운영에는 기능 코드뿐 아니라 전달·실행 과정도 포함된다는 점을 배웠습니다.

4. 어려움 하나 · 재생 안내는 나오는데 음악이 들리지 않았습니다

사용 중 확인한 상황

다른 명령은 동작하고 봇도 음성 채널에 들어왔습니다. `/play`를 입력하면 재생을 시작했다는 안내가 나왔지만 소리가 들리지 않았습니다. `/resume`을 사용해도 해결되지 않았고, 다시 요청해도 같은 상황이 반복됐습니다.

처음에는 버전이나 의존성 문제를 의심했습니다. 하지만 그 추측만으로 라이브러리를 바꾸기보다, 실제로 어디까지 성공했는지 나눠 확인했습니다.

확인한 내용

서버의 음악 처리 로그를 보니 곡 검색은 성공했지만 음원을 가져오는 과정에서 실패하고 있었습니다. 외부 제공자가 로그인을 요구하거나 음원 요청을 거절하는 응답이 남았고, 결국 곡이 불러오기 실패로 끝났습니다.

인증 정보가 환경에 들어 있다는 사실만으로는 충분하지 않았습니다. 인증 갱신이 실제로 성공했다는 로그까지 확인했고, 그 뒤에도 남는 음원 요청 문제를 따로 살폈습니다. 연결, 검색, 인증, 실제 음원 수신을 하나의 성공으로 묶지 않는 것이 중요했습니다.

바꾼 부분

음원 처리 라이브러리의 공개 수정 사항과 요청 방식을 비교해 호환성을 보완했습니다. 또한 봇이 곡을 받았다는 이유만으로 실제 재생까지 성공했다고 안내하지 않도록, 처음에는 **재생 요청을 접수했다**고 알리게 바꿨습니다.

음원 다운로드나 재생 중 나중에 오류가 나면, 원래 요청의 안내도 실패 상태로 바꾸도록 했습니다. 실패 뒤에 늦게 도착한 접수 결과가 실패 안내를 다시 덮어쓰지 않도록 처리한 점도 함께 확인했습니다.

결과와 배운 점

변경 적용 후 실제 음성 채널에서 음악이 정상적으로 나오는 것을 확인했습니다. 동시에 재생이 실패했을 때는 사용자가 성공 안내만 보고 기다리지 않도록 응답 흐름을 개선했습니다.

이 경험 이후에는 컨테이너가 정상이라는 표시나 테스트 통과만으로 음악이 동작한다고 판단하지 않습니다. 실제 `/play`를 사용하고 소리가 들리는지까지 별도로 확인합니다. 외부 음원 서비스가 바뀌면 다시 점검해야 하는 범위도 남아 있습니다.

5. 어려움 둘 · 기능을 나눈 뒤에도 답변이 원래 요청으로 돌아가야 했습니다

구조를 점검하며 확인한 응답 순서

명령을 받는 곳과 음악을 처리하는 곳이 달라지면, 작업이 끝났다는 것만으로 사용자에게 답할 수 있는 것은 아닙니다. 어떤 요청의 결과인지, 어느 답변을 수정해야 하는지 알아야 합니다.

개선 과정에서 작업을 먼저 보내고 답변할 정보를 나중에 저장하던 순서를 살폈습니다. 결과가 매우 빨리 돌아오면, 아직 답변 위치가 기록되지 않은 상황과 겹칠 수 있기 때문입니다.

바꾼 부분

처리 순서를 **응답 준비 → 답변 정보 저장 → 작업 전달**로 바꿨습니다. 작업 결과가 도착하면 저장해 둔 정보를 찾아 원래 요청의 답변을 수정합니다. 개인에게만 보여야 하는 안내도 같은 응답에 이어지도록 했습니다.

이 순서는 주석으로만 남기지 않고 테스트로 확인했습니다. 답변 정보 저장이 작업 전달보다 먼저 일어나는지 검사해, 이후 기능을 고치더라도 순서가 바뀌지 않게 했습니다.

결과와 배운 점

역할을 나눌 때는 요청을 전달하는 경로뿐 아니라 **결과가 사용자에게 돌아오는 경로**까지 함께 설계해야 한다는 점을 배웠습니다. 빠른 처리도 순서가 잘못되면 정상적인 사용자 경험으로 이어지지 않습니다.

답변에 필요한 정보에는 유효 시간이 있으므로, 모든 작업 결과를 언제까지나 복구할 수 있는 구조는 아닙니다. 오래 대기한 곡에서 발생한 오류는 로그로 확인해야 하는 경우가 남아 있습니다.

6. 어려움 셋 · 기능 코드는 그대로인데 빌드가 막혔습니다

배포 과정에서 확인한 상황

음악 라이브러리를 내려받는 과정에서, 외부 저장소가 필요한 설명 파일 대신 웹페이지를 반환하는 문제가 확인됐습니다. 빌드 도구는 라이브러리 설명으로 읽어야 할 내용을 해석하지 못했고, 새 버전을 서버에 전달하는 과정도 막혔습니다.

이때는 기능 코드가 잘못됐다고 가정하기보다 실제로 내려온 파일과 공식 배포 경로를 확인했습니다. 버전을 무조건 올리면 음악 동작까지 함께 바뀔 수 있어, 우선 사용 중인 버전을 유지하는 방향을 택했습니다.

바꾼 부분

같은 버전의 공식 GitHub 배포 파일을 사용하도록 바꾸고, 공식 검증값과 일치하는지 빌드 전에 확인하게 했습니다. 공급받는 경로는 바꾸되, 출처와 파일 확인 절차를 함께 남겼습니다.

이전 운영 과정에서는 별도로 배포 스크립트도 정리했습니다. 반복 배포에서 남는 업로드 파일과 사용하지 않는 이미지를 정리하고, 설정을 검사한 뒤 새 컨테이너의 기동 상태를 기다리도록 했습니다.

결과와 배운 점

변경 이후 자동 빌드와 서버 배포가 성공한 기록을 확인했습니다. 이 경험을 통해 운영에는 기능 구현뿐 아니라 **필요한 파일을 받아 빌드하고, 서버에서 시작한 결과를 확인하는 과정**도 포함된다는 점을 배웠습니다.

현재 배포는 이전 버전을 내리고 새 버전을 올리는 방식입니다. 전환 중 사용에 영향이 있을 수 있어, 배포 뒤에는 기동 확인과 실제 Discord 사용을 구분해 점검합니다.

7. 지금까지 만든 것과 앞으로 확인할 것

구분	현재 범위
구현한 기능	음악 검색·재생·대기열·제어, 모의투자 계좌·매매·순위, 개인 사용법 안내
직접 사용하며 확인한 문제	재생 안내와 실제 무음의 차이, 변경 후 실제 음악 재생
코드·테스트로 확인한 개선	답변 저장 순서, 재생 실패 통지, 늦은 접수 결과가 실패를 덮지 않는 처리
배포 기록으로 확인한 결과	라이브러리 공급 방식 수정 이후 자동 빌드·서버 배포 성공
앞으로 기록을 보강할 부분	실제 이용 지표와 장기간 운영 기록, 배포 뒤 사용자 기능 확인 결과

여기서는 확인할 수 있는 기능과 운영 사례를 중심으로 설명했습니다. 사용자 수의 의미, 테스트와 실제 사용의 차이, 성과로 기재하지 않은 항목은 검증 문서에서 확인할 수 있습니다.

다음에는 기능이 실행되는 것뿐 아니라, 배포 뒤 실제 명령과 음악 재생을 확인한 기록을 더 꾸준히 남기고 싶습니다. 그래야 문제가 다시 생겼을 때 이전 상태와 비교하고, 개선 결과도 더 정확히 설명할 수 있기 때문입니다.

8. 이 프로젝트에서 가져간 경험

DIS에서 가장 크게 배운 것은 새로운 기술을 많이 붙이는 것보다 **사용자가 겪은 상황에서 출발해, 확인할 수 있는 증거를 따라 문제를 좁혀 가는 일**입니다.

음악이 나오지 않을 때는 연결됐다는 표시 대신 실제 음원 요청을 봤고, 기능을 나눌 때는 작업 전달뿐 아니라 답변이 돌아오는 순서를 봤습니다. 배포가 막혔을 때는 기능 코드 밖의 파일 공급과 실행 절차까지 확인했습니다. 앞으로도 직접 사용해 보고, 실패를 확인하고, 바꾼 결과를 설명할 수 있는 서비스를 만들고 싶습니다.

이력서용 요약 · 챌린지 3개 · 주장별 근거 · 참고 사례 분석